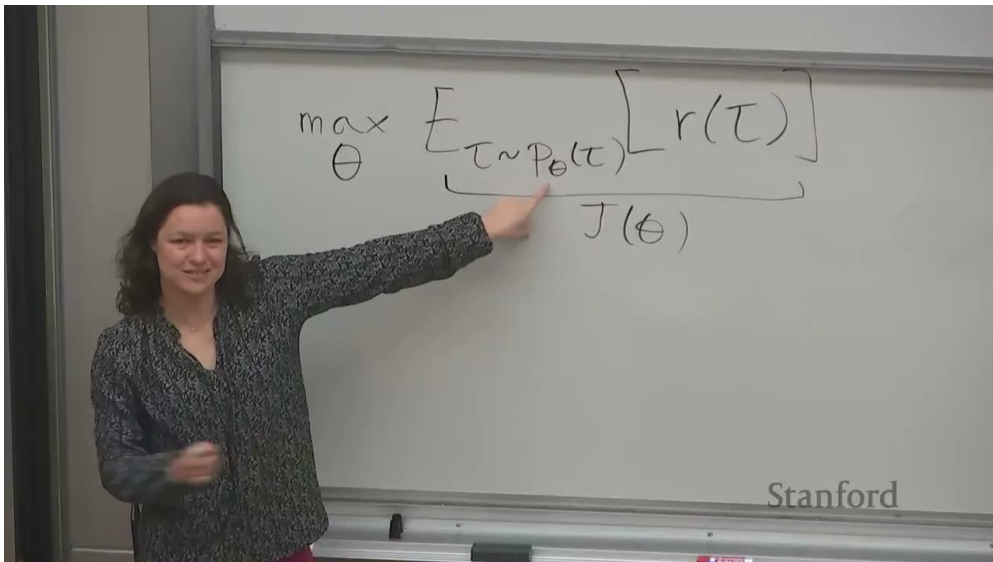


课程笔记

# CS224R 课程笔记: Lecture 3 Policy Gradients

Codex

2026-04-13



视频作者/频道: CS224R / Stanford

发布日期: 课程讲次日期: 2025-04-09

视频时长: 未在本地图元数据中提供

视频链接: <https://www.youtube.com/watch?v=KCAOXd4I09o>

# 目录

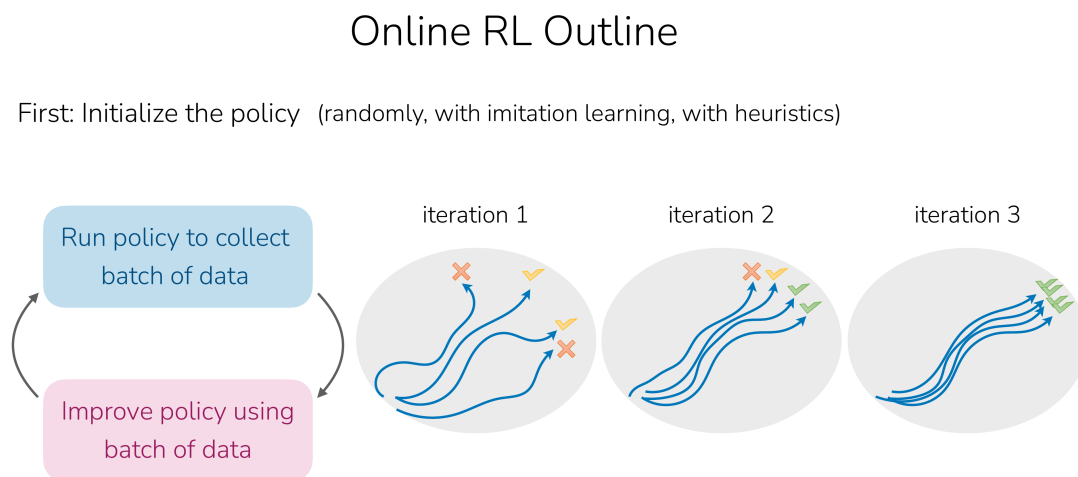
|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>1 从模仿学习走向在线强化学习</b>                                          | <b>2</b> |
| 1.1 在线 RL 的基本循环                                                 | 2        |
| 1.2 目标函数仍然是长期回报                                                 | 2        |
| 1.3 本章小结                                                        | 3        |
| <b>2 Policy Gradient 如何推导出来</b>                                 | <b>3</b> |
| 2.1 从轨迹分布开始写导数                                                  | 3        |
| 2.2 为什么只剩下策略项                                                   | 3        |
| 2.3 用采样近似期望                                                     | 4        |
| 2.4 本章小结                                                        | 4        |
| <b>3 这个梯度到底在做什么</b>                                             | <b>4</b> |
| 3.1 直觉：高回报动作更可能被重复                                              | 4        |
| 3.2 人形机器人行走例子                                                   | 5        |
| 3.3 高方差是 vanilla policy gradient 的核心难点                          | 5        |
| 3.4 本章小结                                                        | 5        |
| <b>4 如何改进：Causality 与 Baseline</b>                              | <b>6</b> |
| 4.1 Causality：动作不应为过去奖励背锅                                       | 6        |
| 4.2 Baseline：只奖励高于平均线的行为                                        | 6        |
| 4.3 本章小结                                                        | 7        |
| <b>5 Off-Policy Policy Gradient、Importance Sampling 与 KL 约束</b> | <b>7</b> |
| 5.1 为什么想做 off-policy                                            | 7        |
| 5.2 Importance Sampling 的基本形式                                   | 7        |
| 5.3 为什么轨迹比值会约化成策略比值                                             | 8        |
| 5.4 KL 约束：别让新旧策略差太远                                             | 8        |
| 5.5 本章小结                                                        | 9        |
| <b>6 总结与延伸</b>                                                  | <b>9</b> |

# 1 从模仿学习走向在线强化学习

第三讲的转折点非常明确：上一讲的 imitation learning 可以高效利用专家演示，但它不能超越专家，也不能通过自己的试错持续改进。于是课程正式进入 online RL，并把 policy gradients 作为第一类在线强化学习算法。

## 1.1 在线 RL 的基本循环

在线强化学习的结构并不复杂，但和离线模仿学习有根本不同。讲者把流程概括成三步循环：初始化策略，运行策略收集数据，再用新数据改进策略，然后重复这一过程。



6

图 1: 在线强化学习的基本循环：初始化策略，roll out 收集数据，再用新数据更新策略。

这里真正发生变化的是数据来源。模仿学习使用静态专家数据；policy gradients 使用的是当前策略自己采样到的轨迹。数据一边被收集，一边又反过来改变策略本身，因此训练是一个闭环过程。

## 1.2 目标函数仍然是长期回报

虽然数据来源变了，但强化学习目标没有变：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[ \sum_{t=1}^T r(s_t, a_t) \right].$$

符号说明如下：

- $\theta$ ：策略参数；
- $p_{\theta}(\tau)$ ：由当前策略和环境动力学共同诱导的轨迹分布；
- $r(s_t, a_t)$ ：每一步奖励；
- $\sum_t r(s_t, a_t)$ ：一条轨迹的总回报。

因此 policy gradients 的任务不是“像专家一样做”，而是直接对这个长期目标做优化。它的吸引力在于概念简单，和优化目标直接对齐。

### 1.3 本章小结

从这一讲开始，课程不再依赖固定专家数据，而是让策略自己与环境交互。online RL 的核心差别在于：数据是当前策略诱导出来的，目标依旧是最大化长期回报，但学习过程已经变成“采样轨迹再改进策略”的循环。

## 2 Policy Gradient 如何推导出来

### 2.1 从轨迹分布开始写导数

讲者先把轨迹回报记作  $r(\tau)$ ，于是目标可以写成

$$J(\theta) = \int p_{\theta}(\tau) r(\tau) d\tau.$$

对它求梯度：

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau.$$

接着使用一个关键恒等式：

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau).$$

于是就得到

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)].$$

这就是 log-derivative trick。它把“对轨迹分布本身求导”的问题，转成了“对轨迹对数概率求导后再做期望”的问题。

### 2.2 为什么只剩下策略项

课上进一步把轨迹分布展开为

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t).$$

其中：

- 初始状态分布  $p(s_1)$  与环境动力学  $p(s_{t+1} | s_t, a_t)$  通常不含  $\theta$ ；
- 真正依赖参数  $\theta$  的，是策略项  $\pi_{\theta}(a_t | s_t)$ 。

因此：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

代回后得到 policy gradient 的常见形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[ \left( \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left( \sum_{t=1}^T r(s_t, a_t) \right) \right].$$

## Estimating the gradient

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[ \left( \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left( \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left( \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left( \sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Full algorithm:

1. sample  $\{\tau^i\}$  from  $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$
2.  $\nabla_{\theta} J(\theta) \approx \sum_i \left( \sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left( \sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3.  $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

“REINFORCE algorithm”, vanilla policy gradient

Slide adapted from Sergey Levine

10

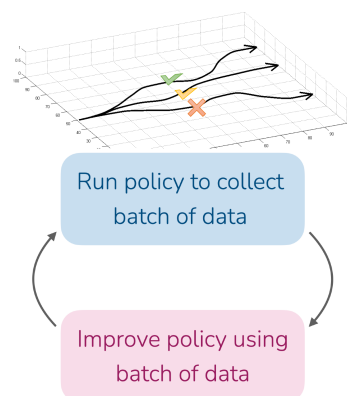


图 2: REINFORCE 的核心估计式：用采样轨迹近似 policy gradient，再沿梯度方向更新策略。

## 2.3 用采样近似期望

因为精确积分通常做不到，课上用 Monte Carlo 采样来近似：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left( \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left( \sum_{t=1}^T r(s_{i,t}, a_{i,t}) \right).$$

这里  $N$  是采样到的轨迹数。然后就可以执行一次梯度上升：

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta).$$

### REINFORCE 的本质

从实现角度看，REINFORCE 就是在每条采样轨迹上，把“动作的对数概率梯度”乘上“整条轨迹的总回报”，再对样本求平均。

## 2.4 本章小结

policy gradient 的推导核心只有两步：先用 log-derivative trick 把对轨迹分布的求导转成对数概率求导，再利用轨迹分布分解掉环境动力学，只保留策略项。这样就把长期回报优化写成了一个可采样估计的梯度。

## 3 这个梯度到底在做什么

### 3.1 直觉：高回报动作更可能被重复

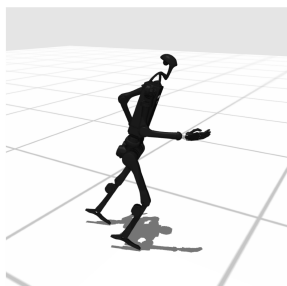
policy gradient 最重要的直觉，是“让高回报轨迹中的动作更可能再次发生，让低回报轨迹中的动作更不可能再次发生”。课程直接把它和 imitation learning 对比：模仿学习是让专家动作更可能，policy gradient 则是让高回报动作更可能。

### 3.2 人形机器人行走例子

讲者用 humanoid walking in simulation 的例子来解释这个梯度。奖励函数取前向速度，因此如果机器人往后倒，奖励可能是负的；如果向前挪动或向前倒，奖励可能是正的。

#### What does the gradient do?

Example: learning humanoid walking in simulation



reward:  $r(\mathbf{s}, \mathbf{a}) = \text{forward velocity of robot}$

(can be negative if robot goes backwards)

1. sample  $\{\tau^i\}$  from  $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$

$\tau^1$ : falls backwards ✗

$\tau^2$ : falls forwards ✓

$\tau^3$ : manages to stand still ✓

$\tau^4$ : one small step forward then falls backwards ✗

$\tau^5$ : one large step backwards then small step forwards ✗

2.  $\nabla_\theta J(\theta) \approx \sum_i (\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)) (\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i))$

Question: what will the gradient encourage the policy to do?

-> will encourage policy to fall forward, and not take step forward 😞

12

Policy gradient is **noisy / high-variance**

图 3: policy gradient 的问题在于：若只用整条轨迹总回报加权，可能会错误鼓励“向前倒下”而不是“真正迈步前进”。

课件展示了几条轨迹：后仰摔倒、前扑摔倒、站着不动、迈一步又摔倒、先大幅后退再小幅前移。由于简单 policy gradient 用的是整条轨迹总回报，它可能把“摔向前方”当成比“向后摔倒”更好的结果，从而鼓励错误行为。

### 3.3 高方差是 vanilla policy gradient 的核心难点

这个例子说明，vanilla policy gradient 的估计非常 noisy / high-variance。问题不在于推导错误，而在于信用分配太粗：

- 一整条轨迹里的所有动作都被同一个总回报加权；
- 早期动作和后期奖励之间的关系容易被错误归因；
- 奖励尺度变化也会让梯度不稳定。

#### 为什么 “do more good stuff” 还不够

因为 “good stuff” 到底对应轨迹中的哪一步动作，并没有被总回报直接分清。回报只是告诉你整条轨迹好不好，却没有自动告诉你是哪一步值得 credit。

### 3.4 本章小结

policy gradient 的直觉非常漂亮，但最原始的实现会遇到严重的高方差问题。它确实会把高回报行为推得更有可能会，但也可能把本不该得 credit 的动作一起推高，因此必须进一步做信用分配和方差控制。

## 4 如何改进：Causality 与 Baseline

### 4.1 Causality：动作不应为过去奖励背锅

第一种改进是 causality。课上的关键观察是：时刻  $t$  的动作不可能影响  $t' < t$  的奖励。因此在给  $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$  分配权重时，应该只看从  $t$  开始往后的 future rewards，而不是整条轨迹的全部奖励。

#### Improving the gradient

Using causality

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left( \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left( \sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$



$\tau^5$ : one large step backwards then small step forwards ✗

Policy behavior at time  $t$  does not affect rewards at time  $t' < t$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left( \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)$$

sum of future rewards

13

图 4: 使用 causality 后，时刻  $t$  的梯度只乘以从  $t$  起的 future rewards，而不是过去已经发生的奖励。

因此梯度估计可以改写成更合理的形式：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left( \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right).$$

这一步并没有改变目标，只是把每个动作对应到它真正可能影响的未来回报区间上。

### 4.2 Baseline：只奖励高于平均线的行为

第二种改进是引入 baseline。讲者给出的直觉非常直接：如果某条轨迹只是“稍微不那么糟”，我们不应该像对待真正优秀轨迹那样强烈鼓励它。于是可以从回报里减去一个基线  $b$ ：

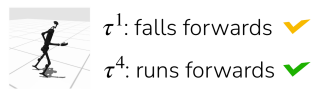
$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b)],$$

其中  $G_t$  可以理解为从时刻  $t$  起的 return。

## Improving the gradient

Introducing baselines

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$



15

图 5: 引入 baseline 后, 梯度更关注 “高于平均” 的行为, 能显著降低方差。

课上强调, 减去一个与动作无关的 baseline 不会在期望上引入偏差, 但能降低方差。最简单的 baseline 可以取 average reward。于是:

- 高于平均线的轨迹给正信号;
- 低于平均线的轨迹给负信号;
- 梯度不再被整体奖励尺度牵着走。

### 4.3 本章小结

causality 解决的是信用分配过粗的问题, 让每一步只对未来负责; baseline 解决的是高方差问题, 让策略只重点强化高于参考线的行为。这两步共同把 vanilla policy gradient 变得更可训练。

## 5 Off-Policy Policy Gradient、Importance Sampling 与 KL 约束

### 5.1 为什么想做 off-policy

vanilla policy gradient 还有一个工程缺点: 它通常是 on-policy 的, 也就是每次更新都依赖当前策略刚采出来的数据。这很浪费, 因为旧数据、历史策略数据都无法直接重用。

于是课程提出问题: 能不能用旧策略甚至其他 proposal distribution 采出来的样本, 来估计新策略的目标?

### 5.2 Importance Sampling 的基本形式

讲者先从一般的 importance sampling 恒等式入手。若目标期望原本在  $p$  下计算, 却只拿到了  $q$  下的样本, 可以写成

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[ \frac{p(x)}{q(x)} f(x) \right].$$



搬到轨迹分布上，就有

$$J(\theta) = \mathbb{E}_{\tau \sim \bar{p}(\tau)} \left[ \frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau) \right],$$

其中  $\bar{p}$  可以是旧策略诱导的轨迹分布。

## Off-policy version of policy gradient?

Importance sampling

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} [r(\tau)]$$

What if we want to use samples from  $\bar{p}(\tau)$ ?  
(e.g. previous policy)

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)} \left[ \frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(s_1)} \prod_{t=1}^T \pi_\theta(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}}{\cancel{p(s_1)} \prod_{t=1}^T \bar{\pi}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}} = \frac{\prod_{t=1}^T \pi_\theta(a_t | s_t)}{\prod_{t=1}^T \bar{\pi}(a_t | s_t)}$$

### Importance sampling

Using proposal distribution  $q$

$$\begin{aligned} E_{x \sim p(x)} [f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)} \left[ \frac{p(x)}{q(x)} f(x) \right] \end{aligned}$$

Note: Important for  $q$  to have non-zero support for high probability  $p(x)$

21

图 6: off-policy policy gradient 的核心工具是 importance sampling: 用 proposal distribution 的样本重加权逼近目标分布。

### 5.3 为什么轨迹比值会约化成策略比值

如果目标策略和采样策略处在同一环境里，那么轨迹概率中的环境动力学项与初始状态分布项会在分子分母中抵消，只剩下策略比值：

$$\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} = \prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_\theta(a_t | s_t)}.$$

这一步非常关键，它说明 off-policy policy gradient 之所以可能，是因为我们虽然不知道环境动力学，但在比值里它被约掉了。

不过讲者也提醒，一个重要前提是 support 问题：proposal 分布  $q$  在高概率目标区域不能为零，否则比值会失效或方差爆炸。

### 5.4 KL 约束：别让新旧策略差太远

即便 importance sampling 理论上成立，如果新旧策略差得太多，权重仍会非常极端，导致估计高方差。为此课上引入 KL constraint 的动机：限制策略更新不要一步跨太远。

可以把这件事理解成一种 trust region 思想。策略当然要变好，但不能瞬间偏离旧策略到一个几乎没有共享支持的区域，否则旧数据就不再可信。

### off-policy 的真实 trade-off

它的优势是数据复用；它的代价是重加权带来的额外方差与稳定性问题。因此 importance sampling 和 KL 约束总是成对出现。

## 5.5 本章小结

off-policy policy gradient 想解决的是 on-policy 的低数据效率问题。importance sampling 允许我们用旧数据估计新策略目标，而轨迹比值在同环境下会约化成策略比值；但如果新旧策略差太大，估计又会变得不稳定，于是需要 KL 约束控制更新步长。

## 6 总结与延伸

第三讲做了两件关键事情。第一，它给出了第一类真正的在线强化学习算法：policy gradients。第二，它没有停留在公式层面，而是把这个方法的直觉、弱点和修正路径一并展示出来。

如果把整讲内容压缩成一条主线，就是：

- 从长期回报目标出发；
- 用 log-derivative trick 推出 policy gradient；
- 用采样轨迹做 Monte Carlo 估计；
- 发现原始估计高方差，于是引入 causality 和 baseline；
- 想复用旧数据时，再引入 off-policy importance sampling 与 KL 约束。

因此，policy gradient 既是一个简单优美的起点，也是一条会自然通向更现代算法的路线。讲者最后也明确预告，后续的 actor-critic 和 PPO 等方法，都会建立在这一讲的思想。

从学习角度看，这一讲最值得反复确认的不是单个公式，而是几个结构性结论：

- policy gradient 优化的是长期目标而不是单步标签；
- 梯度含义是“提升高回报动作概率”；
- 信用分配和方差控制是这类方法的核心工程难点；
- off-policy 数据复用必须和分布校正一起考虑。

理解了这些，再往后看 actor-critic、advantage、trust region 或 PPO，就不会把它们当成一堆新符号，而会知道它们是在继续修正这一讲暴露出的具体问题。